Frontend Cache Management Architecture

Prepared by: Kaushik Mukherjee

Enterprise Architecture Design Document

Version: 1.0
Audience: Frontend Architects, UI Leads, Principal Engineers

---

1. Overview

Purpose

This document describes a scalable, maintainable, and high-performance cache management architecture for enterprise React/Angular/Vue frontend applications.

The design covers

- API Response Caching
- State Cache
- Memory Cache
- Persistent Cache
- Offline Cache
- CDN Cache
- Browser Cache
- Service Worker Cache
- Image Cache
- GraphQL Cache
- Cache Invalidation
- Cache Synchronization
- Multi-tab Cache
- Security Considerations

---

2. Objectives

The cache architecture should provide

✓ Reduced API Calls

✓ Faster UI Rendering

✓ Better User Experience

✓ Offline Support

✓ Scalability

✓ Low Memory Consumption

✓ Easy Cache Invalidation

✓ Predictable Data Flow

✓ Secure Storage

3. **High-Level Architecture**

```
           User
            |
          React UI
            |
   ┌─────────────────┬──────────┐
   |                 |
   ▼                 ▼
 Component Cache      Global Store Cache
(Local State)        (Redux/Zustand)
   |                 |
   └─────────────────┬──────────┘
            |
       Query Cache Layer
   (React Query / RTK Query)
            |
       Memory Cache Layer
            |
      Persistent Cache Layer
   (IndexedDB / LocalStorage)
            |
     Service Worker Cache
            |
         HTTP Cache
            |
          CDN
            |
          Backend
```

4. Cache Layers

Layer 1 – Component Cache

Scope

Single Component

Examples

useMemo() useRef() useState() memo() useCallback()

Used For

- UI State
- Form Values
- Temporary Search
- Pagination

TTL

None

Layer 2 – Global Store Cache

Libraries

- Redux Toolkit
- Zustand
- MobX

Stores

Advantages

- Shared state
- Single source of truth
- Fast lookup

Layer 3 – Query Cache

Libraries

- React Query
- RTK Query
- Apollo Client
- SWR

Stores

API Responses

Pagination

Infinite Scroll

Search Results

Filtered Results

Layer 4 – Memory Cache

Implementation

Map

WeakMap

LRU Cache

HashMap

Suitable For

- Metadata
- Lookup tables
- Dictionary
- Configurations

Advantages

Very Fast

Disadvantages

Lost after refresh

Layer 5 – Persistent Cache

Technologies

- IndexedDB
- LocalStorage
- SessionStorage

Suitable For

Offline Data

Settings

Feature Flags

Translations

User Preferences

Layer 6 – Service Worker Cache

Responsible For

Images

JS

CSS

Fonts

API Responses

Offline Pages

Layer 7 – Browser Cache

Controlled Using

Cache-Control

Expires

ETag

If-Modified-Since

Example - Cache-Control: max-age=86400

Layer 8 – CDN Cache

Example

Cloudflare

AWS CloudFront

Azure CDN

Google CDN

Stores

5. Cache Flow

User Opens Screen

|
▼
Component Cache?
|
Yes → Return
| No
▼ Redux Store?
| Yes → Return
| No
▼ Query Cache?
| Yes → Return
| No ▼ Memory Cache?
| Yes → Return
| No ▼ IndexedDB?
| Yes
| Return
| Background Refresh
▼ API
| Update Every Layer

6. Cache Types

Cache	Lifetime	Storage
Component	Until Unmount	Memory
Redux	Session	Memory
Query	Configurable	Memory
IndexedDB	Permanent	Disk
LocalStorage	Permanent	Disk
SessionStorage	Session	Disk
Service Worker	Configurable	Disk
Browser Cache	HTTP	Browser
CDN	Configurable	Edge

7. Cache Invalidation Strategy

Time Based

TTL = 5 minutes

8. Cache Expiration Policy

Data	TTL
Feature Flags	1 hour
Products	15 min
Dashboard	2 min
User Profile	30 min
Exchange Rate	5 min
Configuration	24 hr
Countries	30 days
States	30 days

9. **Performance Optimizations**

- Lazy Cache Population
- Prefetching
- Background Refresh
- Compression
- Delta Updates
- Batch Fetching
- Pagination Cache
- Infinite Scroll Cache
- Image Lazy Loading
- Virtualization

10. React Implementation Example

```
const cache = new Map();

export async function getProduct(id: string) {
  if (cache.has(id)) {
    return cache.get(id);
  }

  const response = await fetch(`/api/products/${id}`);
  const data = await response.json();

  cache.set(id, data);

  return data;
```

11. Recommended Technology Stack

Purpose	**Recommendation**
Server State	Redux Toolkit Query / React Query
Client State	Redux Toolkit
Memory Cache	LRU Cache
Persistent Cache	IndexedDB
Offline	Service Worker + Workbox
API Cache	RTK Query
Image Cache	Browser + CDN

Purpose	Recommendation
Static Assets	CDN
Compression	Brotli + Gzip

12. Sequence Diagram

```
User
  |
  | Open Dashboard
  ▼
React Component
  |
  | Check Local Cache
  ▼
Cache Manager
  |
  ├── Memory Cache?
  |       |
  |       ├── Yes → Return
  |       |
  |       ▼
  |   IndexedDB?
  |       |
  |       ├── Yes
  |       |     |
  |       |     ▼
  |       | Return Cached Data
  |       |
  |       ▼
  |   API Call
  |       |
  ▼       ▼
Backend
  |
  ▼
Response
  |
  ▼
Update Memory Cache
  |
  ▼
Update IndexedDB
  |
  ▼
Update Redux Store
  |
  ▼
Render UI
```

13. Best Practices

Practice	Recommendation
Single Cache Manager	✅ Yes
Centralized TTL Configuration	✅ Yes
Version Cache Keys	✅ Yes
Namespace Cache Entries	✅ Yes
LRU Eviction	✅ Yes
Background Revalidation	✅ Yes
Stale-While-Revalidate	✅ Yes
Cache Metrics	✅ Yes
Distributed Invalidation	✅ Yes
Cache Compression	✅ Large Payloads
Encrypt Sensitive Cached Data	✅ When Required
Monitor Cache Hit Ratio	✅ Continuously

14. Architecture Decision Matrix

Scenario	Recommended Cache Layer
UI Form State	Component State (useState)
User Session	Redux Toolkit
API Responses	RTK Query / React Query
Large Reference Data (Countries, States)	IndexedDB + Memory Cache
Images	Browser Cache + CDN
Static Assets	CDN + Service Worker
Offline Support	Service Worker + IndexedDB
Feature Flags	Memory + LocalStorage

Scenario	**Recommended Cache Layer**
Application Configuration	IndexedDB + Memory Cache
Real-Time Dashboards	Memory Cache + Background Revalidation
Infinite Scroll	Query Cache + Pagination Cache
Frequently Accessed Metadata	In-Memory LRU Cache